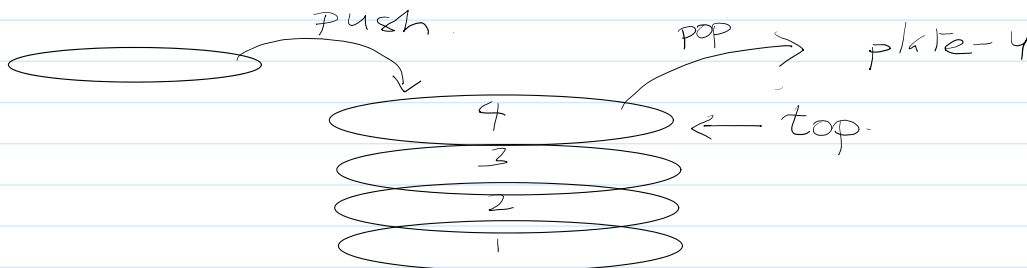


Stack Data Structure

08 August 2026 18:27

A Stack is a linear data structure that follows the LIFO (Last In, First Out) principle. This means:
The element inserted last is the first element to remove.

A real - world example is a stack of plates. If you place plate one above another, you can remove only the plate at the top.



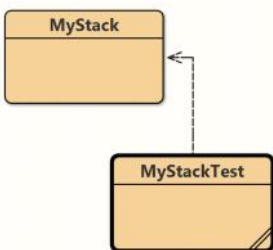
Stack operations

A stack generally provides the following operations:

Operation	Meaning
push()	Insert an element
pop()	Remove the top element
peek()	View the top element without removing it.
isEmpty()	Check whether stack is empty
isFull()	Check whether stack is full
size()	Number of elements

Stack using Array:

We can implement a stack without using a Java's stack class by using a normal array.



```
package DataStructures.StackDataStructure;
```

```
/**
 * Write a description of class MyStack here.
 *
 * @author (your name)
 * @version (a version number or a date)
```

```

*/
public class MyStack
{
    private int stack[];
    private int top, count, capacity;

    public MyStack(int size){
        capacity = size;
        stack = new int[capacity];
        top = -1; //Because initially there are no elements in the stack
        count = 0;
    }

    public MyStack(){
        this(10);
    }

    public void push(int ele){
        if(isFull()){
            System.out.print("\nStack overflow!");
            return;
        }
        stack[++top] = ele;
        count++;
    }

    public boolean isFull(){
        return top == capacity - 1;
    }

    /**
     * @return number of elements present in the current MyStack object
     */
    public int numberOfElements(){
        return count;
    }

    public int pop(){
        if(isEmpty()){
            return Integer.MIN_VALUE;
        }
        int v = stack[top--];
        count--;
        return v;
    }

    public int peek(){
        if(isEmpty()){
            return Integer.MIN_VALUE;
        }

        return stack[top];
    }
}

```

```

    }

    public boolean isEmpty(){
        return top == -1;
    }

    public int capacityOfStack(){
        return capacity;
    }
}

```

```

package DataStructures.StackDataStructure;
import java.util.*;

```

```

/**
 * Write a description of class MyProgram here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class MyStackTest
{
    public static Scanner sc = new Scanner(System.in);
    public static void main(String[] args){
        System.out.print("\f");
        int size;
        System.out.print("\nEnter the size of your stack: ");
        size = sc.nextInt();
        MyStack s = new MyStack(size);

        while(true){
            System.out.print("\n=====");
            System.out.print("\n=====Menu=====");
            System.out.print("\n=====");
            System.out.print("\n1. Push\n2. Pop\n3. Peek"+
                "\n4 Number of elements\n5 Capacity of Stack\n6 Exit"+
                "\nEnter your choice: ");
            int ch = sc.nextInt();
            switch(ch){
                case 1:
                    System.out.print("\nEnter integer value to push into Stack: ");
                    int e = sc.nextInt();
                    s.push(e);
                    break;
                case 2:
                    e = s.pop();
                    if(e == Integer.MIN_VALUE){
                        System.out.print("\nStack Underflow!");
                    }
                else{
                    System.out.print("\nPopped element: " + e);

```

```

        }
        break;
    case 3:
        e = s.peek();
        if(e == Integer.MIN_VALUE){
            System.out.print("\nStack is empty!");
        }
        else{
            System.out.print("\nPeak element: " + e);
        }
        break;
    case 4:
        System.out.print(
            "\nNumber of elements present in the Stack: "+
            s.numberOfElements());

        break;
    case 5:
        System.out.print("\nCapacity of stack: "+
            s.capacityOfStack());
        break;
    case 6:
        System.out.print("\nGood bye");
        System.exit(ch);
    default:
        System.out.print("\nWrong menu item selected.");
    }
}
}
}
}
}

```

Output: Execute the code in your system.

Applications of Stack:

Stacks are widely used in computer science.

1. **Function Call:** When a method calls another method, Java uses a call stack to keep the track of method execution.

```

main()
|
V
method1()
|
V
method2()

```

When method2() finishes execution, returns to method1()

2. **Undo/Redo:** The text editors can maintain previous operations using Stack. Undo removes the most recent operation first.

3. **Browser History:** A stack like structure can be used to manage navigation history.

4. **Expression Evaluation:** Stacks are used to evaluate expressions such as $10 + 20 * 30$
5. **Parentheses Matching:** A stack can check whether parentheses are balanced.
[[()]]
The expression is balanced
For: {[()]} => This expression is not balanced.
6. **Depth-first Search (DFS):** DFS for graphs and trees can be implemented using a stack.
7. **Backtracking:** Stack structures are useful in problems such as:
 - a) Maze solving
 - b) Sudoku
 - c) Path finding
 - d) Backtracking algorithms